

Import and display the first few rows

data.head()

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	9.1
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29:00 AM	Cash	76.40	4.761905	3.8200	9.6
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	1:23:00 PM	Credit card	324.31	4.761905	16.2155	7.4
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	8.4
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	5.3

Next steps: [Generate code with data](#) [New interactive sheet](#)

Convert the Date column to datetime format

data['Date'] = pd.to_datetime(data['Date'])
data.head(1)

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	2019-01-05	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	9.1

Next steps: [Generate code with data](#) [New interactive sheet](#)

Create a new column Month extracted from the Date column.

data['Month'] = data['Date'].dt.month_name()
data

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating	Month
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	2019-01-05	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	9.1	January
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	2019-03-08	10:29:00 AM	Cash	76.40	4.761905	3.8200	9.6	March
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	2019-03-03	1:23:00 PM	Credit card	324.31	4.761905	16.2155	7.4	March
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	2019-01-27	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	8.4	January
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2019-02-08	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	5.3	February
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	233-67-5758	Giza	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.0175	42.3675	2019-01-29	1:46:00 PM	Ewallet	40.35	4.761905	2.0175	6.2	January
996	303-96-2227	Cairo	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.6900	1022.4900	2019-03-02	5:16:00 PM	Ewallet	973.80	4.761905	48.6900	4.4	March
997	727-02-1313	Alex	Yangon	Member	Male	Food and beverages	31.84	1	1.5920	33.4320	2019-02-09	1:22:00 PM	Cash	31.84	4.761905	1.5920	7.7	February
998	347-56-2442	Alex	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.2910	69.1110	2019-02-22	3:33:00 PM	Cash	65.82	4.761905	3.2910	4.1	February
999	849-09-3807	Alex	Yangon	Member	Female	Fashion accessories	88.34	7	30.9190	649.2990	2019-02-18	1:28:00 PM	Cash	618.38	4.761905	30.9190	6.6	February

Next steps: [Generate code with data](#) [New interactive sheet](#)

Check and remove duplicate rows

duplicates = data.duplicated().sum()
duplicates

np.int64(0)

data.head()

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating	Month
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	2019-01-05	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	9.1	January
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	2019-03-08	10:29:00 AM	Cash	76.40	4.761905	3.8200	9.6	March
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	2019-03-03	1:23:00 PM	Credit card	324.31	4.761905	16.2155	7.4	March
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	2019-01-27	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	8.4	January
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2019-02-08	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	5.3	February

Verify and correct data types of all columns.

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   object
1   Branch                1000 non-null   object
2   City                  1000 non-null   object
3   Customer type         1000 non-null   object
4   Gender                1000 non-null   object
5   Product line          1000 non-null   object
6   Unit price            1000 non-null   float64
7   Quantity              1000 non-null   int64
8   Tax 5%                1000 non-null   float64
9   Sales                 1000 non-null   float64
10  Date                  1000 non-null   datetime64[ns]
11  Time                  1000 non-null   object
12  Payment               1000 non-null   object
13  cogs                  1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income          1000 non-null   float64
16  Rating                1000 non-null   float64
17  Month                 1000 non-null   object
dtypes: datetime64[ns](1), float64(7), int64(1), object(9)
memory usage: 140.8+ KB
```

In the dataset, the **Time** column should be converted from object to time format, and the **Month** column should be categorical or integer based on its values. Columns like **Branch**, **City**, **Customer type**, **Gender**, **Product line**, and **Payment** are categorical but stored as objects, so they should be changed to the **category** data type for accuracy and efficiency.

Data Type Corrections Required

```
# Convert to category (for categorical columns)
cat_cols = ['Branch', 'City', 'Customer type', 'Gender', 'Product line', 'Payment', 'Month']
data[cat_cols] = data[cat_cols].astype('category')

# Convert Time column to time format
data['Time'] = pd.to_datetime(data['Time'], format='%H:%M:%S').dt.time
```

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   object
1   Branch                1000 non-null   category
2   City                  1000 non-null   category
3   Customer type         1000 non-null   category
4   Gender                1000 non-null   category
5   Product line          1000 non-null   category
6   Unit price            1000 non-null   float64
7   Quantity              1000 non-null   int64
8   Tax 5%                1000 non-null   float64
9   Sales                 1000 non-null   float64
10  Date                  1000 non-null   datetime64[ns]
11  Time                  1000 non-null   object
12  Payment               1000 non-null   category
13  cogs                  1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income          1000 non-null   float64
16  Rating                1000 non-null   float64
17  Month                 1000 non-null   category
18  Day                   1000 non-null   object
19  Is_Weekend            1000 non-null   bool
20  DateTime              1000 non-null   datetime64[ns]
21  Hour                  1000 non-null   int32
22  Rating_Category       1000 non-null   object
23  Profit_Margin         1000 non-null   float64
dtypes: bool(1), category(7), datetime64[ns](2), float64(8), int32(1), int64(1), object(4)
memory usage: 130.0+ KB
```

Basic Data Analysis

1.How many unique cities and branches are in the dataset?

```
data.groupby('Branch')['City'].unique()
```

City	
Branch	
Alex	[Yangon]
Cairo	[Mandalay]
Giza	[Naypyitaw]
dtype: object	

2.Which branch has the highest total gross income?

data.groupby('Branch')['gross income'].sum().head(1)	
gross income	
Branch	
Alex	5057.1605
dtype: float64	

3.What is the average rating per branch?

data.groupby('Branch')['Rating'].mean()	
Rating	
Branch	
Alex	7.027059
Cairo	6.818072
Giza	7.072866
dtype: float64	

4.Which is the most common payment method?

data['Payment'].value_counts().head(1)	
count	
Payment	
Ewallet	345
dtype: int64	

5.Find the total sales for each product line

data.groupby('Product line')['Sales'].sum()	
Sales	
Product line	
Electronic accessories	54337.5315
Fashion accessories	54305.8950
Food and beverages	56144.8440
Health and beauty	49193.7390
Home and lifestyle	53861.9130
Sports and travel	55122.8265
dtype: float64	

6.Which gender contributes more to the total sales?

data.groupby('Gender')['Sales'].sum().head(1)	
Sales	
Gender	
Female	194671.8375
dtype: float64	

7.What is the average gross income per customer type?

data.groupby('Customer type')['gross income'].mean()	
--	--

gross income	
Customer type	
Member	15.987759
Normal	14.589161
dtype: float64	

8.How many invoices were generated in February 2019?

```
feb_invoices = data[data['Date'].dt.month == 2]
print("feb_invoices are :",feb_invoices.shape[0])
```

```
feb_invoices are : 303
```

9.Which product line has the highest average rating?

```
data.groupby('Product line')['Rating'].mean().sort_values(ascending=False).head(1)
```

Rating	
Product line	
Food and beverages	7.113218
dtype: float64	

10.Which branch sold the highest quantity of items?

```
data.groupby('Branch')['Quantity'].sum().sort_values(ascending=False).head(1)
```

Quantity	
Branch	
Alex	1859
dtype: int64	

Intermediate-Level Tasks

- 1.Create a new column Revenue_Category based on the Total amount: - If Total > 700 → “High”
- If 400 ≤ Total ≤ 700 → “Medium” -> Else → “Low”

Find how many transactions fall under each category.

```
data['Revenue_category'] = pd.cut(data['Sales'], bins=[0, 400, 700, float('inf')], labels=['Low', 'Medium', 'High'])
data.head()
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating	Month	Revenue_category
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	2019-01-05	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	9.1	January	Medium
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	2019-03-08	10:29:00 AM	Cash	76.40	4.761905	3.8200	9.6	March	Low
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	2019-03-03	1:23:00 PM	Credit card	324.31	4.761905	16.2155	7.4	March	Low
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	2019-01-27	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	8.4	January	Medium
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2019-02-08	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	5.3	February	Medium

Next steps: [Generate code with data](#) [New interactive sheet](#)

- 2.Create a column Is_Weekend that is True if the transaction date was on Saturday or Sunday, otherwise False.

Find the percentage of total sales made on weekends.

```
data['Day'] = data['Date'].dt.day_name()
```

```
def weekend(a):
    if a == 'Saturday' or a == 'Sunday':
        return True
    else:
        return False
```

```
data['Is_Weekend'] = data['Day'].apply(weekend)
```

```
weekend_sales = data[data['Is_Weekend'] == True]['Sales'].sum()
total_sales = data['Sales'].sum()
percentage_weekend_sales = round((weekend_sales / total_sales) * 100,2)
print("percentage_weekend_sales is :",percentage_weekend_sales)
```

percentage_weekend_sales is : 31.14

Multi-Level Aggregation

1.Group by both Branch and Gender to find: - Total sales - Average rating - Count of invoice

```
data.groupby(['Branch','Gender'])[['Sales','Rating']].agg(['sum','mean','count'])
```

		Sales		Rating			
		sum	mean	count	sum	mean	count
Branch	Gender						
Alex	Female	63277.7565	338.383725	187	1291.5	6.906417	187
	Male	42922.6140	280.539961	153	1097.7	7.174510	153
Cairo	Female	61210.0860	332.663511	184	1254.6	6.818478	184
	Male	44987.5860	303.970176	148	1009.0	6.817568	148
Giza	Female	70183.9950	350.919975	200	1427.9	7.139500	200
	Male	40384.7115	315.505559	128	892.0	6.968750	128

2.For each month, find the most sold product line (based on quantity).

```
most_sold_product_per_month = data.groupby('Month')['Product line'].agg(pd.Series.mode)
print(most_sold_product_per_month)
```

Month
February Food and beverages
January Sports and travel
March Home and lifestyle
Name: Product line, dtype: object

String & Data Extraction

1. Extract the hour from the Time column. Determine during which hour the supermarket receives the most invoices.

```
data['DateTime'] = pd.to_datetime(data['Date'].astype(str) + ' ' + data['Time'])
data['Hour'] = data['DateTime'].dt.hour
```

/tmp/ipython-input-4027627577.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent
data['DateTime'] = pd.to_datetime(data['Date'].astype(str) + ' ' + data['Time'])

```
data.groupby('Hour')['Invoice ID'].count().sort_values(ascending=False).head(1)
```

Invoice ID	
Hour	
19	113

dtype: int64

2.Find the average gross income for customers who are Members and used Ewallet as payment.

```
cus_data = data[(data['Customer type'] == 'Member') & (data['Payment'] == 'Ewallet')]
avg_gross_income=cus_data['gross income'].mean().round(2)
print("The average gross income for Members using Ewallet as payment is:", avg_gross_income)
```

The average gross income for Members using Ewallet as payment is: 15.83

Advanced Filtering

1.Display all transactions where: - Product line is “Health and beauty” - Rating is above 8 - Payment is “Credit card” Show the top 5 such records sorted by Total.

```
data[(data['Product line'] == 'Health and beauty') & (data['Rating'] > 8) & (data['Payment'] == 'Credit card')].sort_values(by='Sales', ascending=False).head()
```


	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	...	cogs	gross margin percentage	gross income	Rating	Month	Revenue_category	Day	Is_We
906	450-42-3339	Giza	Naypyitaw	Normal	Male	Health and beauty	84.61	10	42.3050	888.4050	...	846.10	4.761905	42.3050	8.8	February	High	Saturday	
989	430-53-4718	Cairo	Mandalay	Member	Male	Health and beauty	75.37	8	30.1480	633.1080	...	602.96	4.761905	30.1480	8.4	January	Medium	Monday	

2.Find which city had the highest average gross income per invoice.

```
data.groupby('City')['gross income'].mean().sort_values(ascending=0).head(1)
```

	gross income
City	
Naypyitaw	16.052367

dtype: float64

Custom Functions

1. Write a custom function to categorize customers based on Rating: - 8–10 → “Excellent” - 6–7.9 → “Good” - Below 6 → “Average” Apply this function to create a new column Rating_Category and count how many customers fall into each category.

```
def cust(cat):
    if cat >= 8 and cat <= 10:
        return 'Excellent'
    elif cat >= 6 and cat <= 7.9:
        return 'Good'
    else:
        return 'Average'
```

```
data['Rating_Category'] = data['Rating'].apply(cust)
```

```
data['Rating_Category'].value_counts()
```

	count
Rating_Category	
Good	356
Excellent	329
Average	315

dtype: int64

2.For each product line, find: - Minimum, maximum, and average total sales - Number of unique payment methods used

```
data.groupby('Product line').agg({'Sales': ['min', 'max', 'mean'], 'Payment': lambda x: x.nunique()})
```

Product line	Sales			Payment
	min	max	mean	<lambda>
Electronic accessories	26.7225	942.4485	319.632538	3
Fashion accessories	12.6945	1042.6500	305.089298	3
Food and beverages	22.6590	1034.4600	322.671517	3
Health and beauty	18.6375	950.2500	323.643020	3
Home and lifestyle	14.6790	1023.7500	336.636956	3
Sports and travel	10.6785	1002.1200	332.065220	3

Task :

Create a new column: Profit Margin (%) = (gross income / Total) * 100. Find which product line has the highest average profit margin.

```
data['Profit_Margin']=data['gross income']/data['Sales']*100
```

```
data.groupby('Product line')['Profit_Margin'].mean().sort_values(ascending=False).head(1)
```

Product line	Profit_Margin
--------------	---------------